当代人工智能实验课　实验二 搜索算法

姓名：张凌琨　　学号：10241900443

实验目标与研究问题

实现 A*、Minimax 和 Alpha-Beta 后，我用四个正式样例核对答案。接着比较访问量和总时间，检查同一改动在不同地图和树上是否仍有效。遇到异常结果，我先固定输入和其他设置，再改变一个条件寻找原因。

输入输出与实验方法

四题输入输出约定

坐标从 0 开始，按(row,col)书写。题一、二的文本依次为起点、终点和二维列表；题一字符加引号，题二 S/E/X 可不加。均支持--input 与--stdin。

题目	主要输入内容	正式输出
一	S/E/./X 字符矩阵	含起终点的路径；无解[]
二	非负整数及 S/E/X 矩阵	最小进入费；E 免费，无解 −1
三	JSON 树，内部 children、叶 value；可包 tree_structure	MAX 根值，逐层交替 MAX/MIN
四	JSON 的 grid 与 treasures 数组；元素含 id、position、value	样例 6×6 矩阵及 H2/H4 首步、U

表 1 四题输入结构与输出格式

题四从地图标记或 starts 读取 A/B，--depth 默认比较 2/4。四题省略输入时运行内置样例。

算法伪代码

g 记最小已知到达费，h 为剩余费用下界，f=g+h。堆按 f 取最小项，同分默认 FIFO，low_h 先取较小 h。严格松弛允许改善后重新入堆，过期项出堆时跳过。

```
算法 1 A*(S,E)
g[S]←0; 其余 g←∞; parent←{}
OPEN←{(h(S),S,0)}
while OPEN≠∅:
  (f,u,gu)←pop_min(OPEN)
  if gu≠g[u]: continue
  if u=E: return 回溯(parent,E),gu
  for v in 合法邻格(u):
    a←gu+c(u,v)
    if a<g[v]:
      g[v]←a; parent[v]←u
      push(OPEN,(a+h(v),v,a))
return no_path
```

题一、二无解分别输出[]和−1，题四不可达为∞。题三 Actions 为孩子序列，Next 取孩子并换玩家，忽略 d，到叶返回叶值。题四按算法 6 转移，深度为 0 或无合法动作时返回 U。op 在 MAX 取 max，在 MIN 取 min，init 分别为−∞与+∞。

```
算法 2 Minimax(s,d)
if Stop(s,d): return Eval(s)
v←init(s)
for a in Actions(s):
  v←op(v,Minimax(Next(s,a),d-1))
return v
```

```
算法 3 Alpha-Beta AB(s,d,α,β)
if Stop(s,d): return Eval(s)
v←init(s)
for a in ordered Actions(s):
  v←op(v,AB(Next(s,a),d-1,α,β))
  if MAX: α←max(α,v)
  else:   β←min(β,v)
  if α≥β: break
return v
```

α 是 MAX 当前可保证的下界，β 是 MIN 当前可保证的上界。α ≥ β 时可跳过余下分支。根窗口为(−∞,+∞)，内部截断可能只返回界。

核心搜索复杂度

V、E 为顶点数和边数，b 为最大分支数，H 为搜索深度。辅助空间和时间以搜索核心为口径，输入、验证、排序、轨迹与路径复制另计。

算法	时间	辅助空间
A* 二叉堆	O((V+E) log V)	一般 O(V+E)，四邻 O(V)
Minimax	O(b^H)	DFS 栈 O(H)
Alpha-Beta 最坏	O(b^H)	DFS 栈 O(H)
Alpha-Beta 理想	O(b^⌈H/2⌉)	DFS 栈 O(H)

表 2 核心算法的理论复杂度

A*假定简单图、估价 O(1)且一致。每边至多扫描一次，懒删堆保留旧项，四邻 E=O(V)。不一致启发若重开，按实际扫描 A 次、入堆 P 次计 O(A+P log P)。DFS 空间只计递归帧。

均匀 b 叉树在理想子序下读取 b^⌈H/2⌉+b^⌊H/2⌋−1 个叶，偶数 H 时为 O(b^(H/2))。好候选先收紧窗口，余下分支只需给出足以截断的界。

随机生成与控制实验设计

手工反例定位机制，固定种子新图和新树检验适用范围。下表按输入统计，重复计时另计。

研究组	生成与固定条件	对照目的
题一	3 尺寸×5 障碍率×20 种子=300 记录；p=0 为重复基线	同图改 h/排序；有解无解分层
题二	200 个不同地图—端点组合，含随机费用与零权；桶另设 12 图×7 倍率	同图免费 E；下界/排序/容器消融
题三	Scout 含 18 棵新随机树及 4 组镜像结构；共享度另设组	同树同序；计入全部探测重搜
题四	30 个固定种子跨图输入；缓存另用 4/6/8 宝藏各 6 新输入	同深度 MM/AB、顺序和缓存控制

表 3 各研究组的生成规则与控制变量

Windows 11、Python 3.12.14。预热后随机交错计时。桶、Scout、增量缓存均为 7 块，每块分别测 2/3/1 次；题四样例和价值扫描 7 块，跨图排序和延迟查询 5 块，均每块 3 次。报告块均值的中位数及 IQR。

用 BFS/Dijkstra 核对路径费用，完整枚举核对博弈。扩展不含目标，博弈访问含根。239 项测试通过。

题目一 迷宫寻路

最短路径怎样得到

题一以当前位置(r,c)为状态，每次上下左右移一格，边代价均为 1。曼哈顿距离 h(n)=|r−rE|+|c−cE| 是剩余步数下界：一步最多使行列差的绝对值之和减少 1，障碍只可能迫使绕行，不能缩短这个必要距离。相邻格的 h 最多相差 1，所以还满足 h(n)≤1+h(n′)，即一致性。

正式样例返回路径为：

```
(0,0) → (0,1) → (0,2) → (1,2) →
(1,3) → (1,4) → (2,4) → (3,4)
```

路径包含 8 个坐标，对应 7 次移动。起终点曼哈顿距离也是 7，因此该路径达到下界；独立 BFS 同样得到 7。沿这条路径，g 从 0 增到 7，h 从 7 降到 0，f 始终为 7。

下面在相同输入上比较 h=0、曼哈顿/FIFO、曼哈顿/low_h。h=0 使 A*退化为一致代价搜索；本题单位费用下与 BFS 按相同距离层次搜索，但仍使用堆，运行开销不必相同。

场景	最短步数	h=0/FIFO 扩展	曼哈顿/FIFO 扩展	曼哈顿/low_h 扩展
PDF 样例	7	15	13	7
开阔 40×40	78	1599	1599	78
强制绕路	64	588	527	318

表 4 相同地图下的启发与同分策略

长墙使曼哈顿 20 的查询绕行至 64 步，FIFO 仍扩展 527 格。另测固定无解图，三种设置均遍历起点所在的 105 格连通区域，启发函数未减少这部分扩展。

为什么准确启发仍会搜索很多

开阔 40×40 图上，曼哈顿已等于真实剩余距离，FIFO 仍和 h=0 一样扩展 1599 格。我固定地图、端点和 h，只把同 f 排序改成 low_h。每格最优 g=r+c，h=78−r−c，f 恒为 78，主键无法区分候选。FIFO 沿波前推进，low_h 优先取 h 更小的格子，扩展降至 78 格。

开阔图换用 low_h 后，时间中位数从 4.52516 降至 0.37323ms，堆峰值却从 40 增至 77：沿一支深入时，其他候选仍留在堆里。正式样例固定 FIFO、仅换启发，扩展 15→13，时间差不足 0.001ms。

地标数量与总运行时间

我用 BFS 预计算地标距离，以 max(曼哈顿, |d(L,n)−d(L,E)|)作下界，由三角不等式保证可采纳。扫描地标数后固定 k=0/1/2，重采 15 块，均计入建表和 40 次查询。

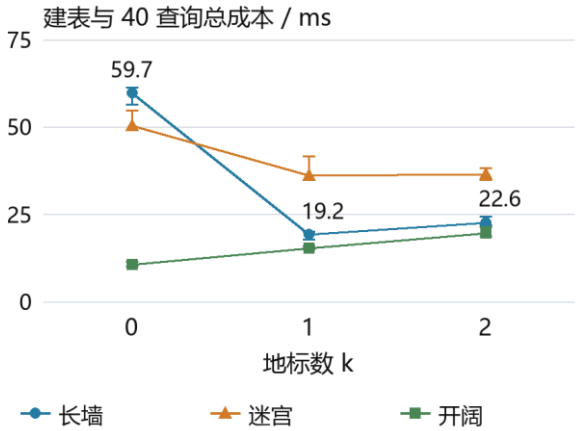


图 1 建表与 40 查询总成本。点为 15 块中位数，竖线为四分位区间 [Q1,Q3]

长墙 k=0/1/2 总耗时 59.703/19.162/22.629ms。第一个地标使扩展 18483→2068，第二个只增加开销，k=1 在 15 块均胜两侧。迷宫 k=1 与 2 的配对区间跨 0，开阔图扩展相同且不建表最快。复验用原三张图，低点限于所测长墙。

走廊压缩是否与地标互补

把度 2 走廊内部收缩为等长边，查询时接回端点并还原格点路径。宏边保距且不短于端点曼哈顿，h 仍一致。建图 O(RC+V+E)，另计路径输出。

18 图各做 1/4/16/64 查询。64 次时压缩/原版 low_h 总耗时比中位为迷宫 0.155、长廊 0.067、随机障碍 1.119、开阔 1.819。61×61 迷宫同模块不开/开压缩为 191.964/29.014ms，压缩加地标为 25.224ms。迷宫所测 4 或 16 次开始回本，开阔至 64 次仍慢。全部 1152 查询经 BFS 核对最短路。

题目二 加权地形

逐格收费与免费终点

题二最小化费用而非步数。边 u→v 的代价是进入 v 的权值，起点初始 g=0；按题目规定，进入 E 的最后一步收费 0。正式样例的一条最优路径及费用如下。

次序	坐标	进入费用	累计费用
0	(0,0)	0	0
1	(1,0)	1	1
2	(2,0)	1	2
3	(3,0)	1	3
4	(3,1)	1	4
5	(3,2)	1	5
6	(4,2)	1	6
7	(4,3)	1	7
8	(4,4)	0	7

表 5 题二样例路径的逐格进入费用

路径共 8 步，7 次进入普通格各付 1，最后免费，总费 7。低费绕行可能更便宜，按步数找路无法比较进入费。

曼哈顿距离是否仍可采纳

原始曼哈顿距离在本题中不可采纳。若所有进入费都至少 1，M=Manhattan 可作下界；但本题进入 E 免费，邻 E 状态的真实剩余费用为 0 而 M=1，已经高估。

令 m 为普通可通行格最低进入费，取下界：

$$h_c(n) = m \times \max(M(n,E) - 1, 0)$$

任意路径至少走 M 步，其中最后一次进入免费，其余至少 M−1 次收费不低于 m。相邻非终点边上 h_c 至多下降 m，不超过进入费用；进入 E 前 h_c 已为 0，因此免费边也满足一致性。如果普通格含 0 权，m=0，h_c 安全退化为零启发。

为检验高估会不会改变答案，我用上排 S、1、E 和下排 0、0、0 构造 2×3 网格。直达收费 1，绕下排收费 0。原始曼哈顿高估绕行的余费并提前返回 1，校正版得到 0。这个零权反例补足了 PDF 正权样例无法检验的情形。

普通费均至少 1 时，M 与 max(M−1,0)在非终点只差常数 1，样例仍可答对。加入 0 权后，在 29 个有解随机输入中，原 M 错 11 例，校正启发全部正确。

把地形结构加入下界

全局 m 忽略高费位置。我把到目标所跨行、列的最小进入费加总为 h_{rc}。一步只跨一行或一列，估计的下降不超过实际进入费，因而一致。

行列下界还可以收紧：它重复使用了一次免费终点。目标行和目标列都因含 E 而最小费为 0；若当前位置行列都未对齐，路径必须先进入目标行或目标列，再到达二者交点 E。这两次进入不可能都免费，至少一次目的格不是 E，费用至少 m。因此定义收紧版本：

$$B(n) = h_{rc}(n) + m \times I[r \neq r_E \text{ 且 } c \neq c_E]$$

$$h_t(n) = \max(h_c(n), B(n))$$

首次对齐一轴时，进入该行或列原计费为 0。该步尚未到 E，实际至少付 m，能覆盖指示项的下降。进入 E 前后指示项均为 0，所以 h_t 仍一致，预处理后常数次查表。

我用最优费用为 83 的 31×31 高费行列带检验收紧效果。固定 low_h 时，校正、行列和收紧下界的起点估计为 59、82、83，扩展分别为 850、811、60。我再换回 FIFO，并按 f 相对最优费用 C*的位置拆分节点数，检查这一下降依赖哪些条件。

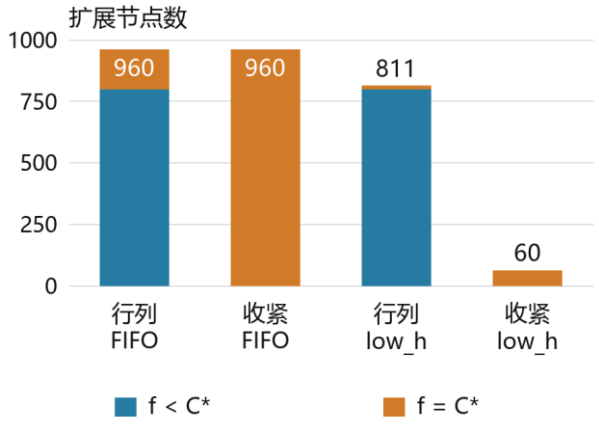


图 2 启发强度与同分排序共同决定最优 f 平台上的扩展

相同 FIFO 下，row_column 有 800 个 f<C* 状态和 160 个 f=C* 状态，总计 960；tight 没有 f<C* 扩展，却仍有 960 个 f=C* 扩展，总数不变。换成 low_h 后，前者为 800+11=811，后者为 0+60=60。更强下界消除了最优边界之前的扩展，也可能把很多状态集中到边界；是否继续访问旁支由同分次序决定。

收益来自下界与排序的共同作用。普通 0 权使 m=0，tight 附加项消失；均匀正权图上几个安全下界也可能给出相同节点数。基础同轮高费带 corrected/tight 为 4.443/0.823ms，零权高费带虽少扩展却可能更慢。

整数队列与费用单位归一化

一致整数 h 使 d=c(v)+h(v)−h(u)非负。我用 D=max d 构造 D+1 个循环桶，桶内沿用同分规则，超过 32768 槽便回退到堆。归一化以正进入费的 gcd 换算桶下标，费用账本仍用原单位。

我在 12 张图上把普通费乘 q=1、2、4、8、16、64、256，E 保持免费。堆与桶的路径、有效出队次序一致，便可单独观察费用范围的影响。计时含 h 表、gcd 和 D 扫描，并对照同模块堆及原 tight+low_h。

31×31 零权随机图取 q=256 时，槽数 1793→8，空扫 13568→53。两种桶均扩展 836 次，耗时 14.046→8.907ms，原版为 6.090ms。三张零权图的归一化/原桶耗时比中位在 q=256 时为 0.657，在 q=1 时为 1.031。168 组配对比较均未快过原版，正式程序沿用原堆。

题目三 极大极小博弈树的最佳得分

样例回传与实际剪枝

正式 JSON 中的四个 MAX 子树叶对为(3,5)、(6,9)、(6,9)、(1,2)，完整子树值为 5、9、9、2。两个 MIN 分别回传 5 和 2，根 MAX 得到 5。题目示意图较简略，输入实现以完整 JSON 为准；Python 示例中的共享对象也保持同一展开博弈值。

左 MIN 的第一个孩子已返回 5，因此其后 MAX 的 β=5。该 MAX 读到 6 后已有 α=6≥β=5，另一个叶 9 可以跳过，返回 6。这里 6 只说明该子树值至少为 6，完整真值仍为 9；左 MIN 已经有更好的 5，所以不需要知道更大的确切值。

Minimax 共访问 15 次、读取 8 叶，左到右 Alpha-Beta 访问 14 次、读取 7 叶。样例只省一个叶，策略准备与搜索时间中位约 0.0389/0.0393ms，几乎相当。右 MIN 在最后一个孩子后也可能发生窗口交叉，但没有兄弟可省，因此“界交叉次数”与“有效剪枝次数”分开记录。

顺序怎样影响搜索与准备成本

我固定同一棵树和叶值，只改变子节点顺序，观察剪枝发生的位置和省下的访问量。真值优先序和逆序先完整求值，用来查看顺序可能带来的差距，准备时间也计入总成本。下表用分支数 4、深度 6、种子 20261030 的树。各方法根值一致。

方法	搜索访问	搜索叶	准备真值叶读取	策略准备与搜索中位 ms
Minimax 左序	5461	4096	0	11.626
Alpha-Beta 左序	1817	1181	0	4.989
Alpha-Beta 右序	700	451	0	1.843
Alpha-Beta 固定随机	1422	915	0	8.539
Alpha-Beta 真值优先	268	127	4096	7.482
Alpha-Beta 真值逆序	4720	3462	4096	22.915

表 6 同一棵树的访问顺序与策略成本

真值优先把搜索叶数从左序的 1181 降到 127，事前却要读 4096 叶。计入这部分开销，真值优先总耗时为 7.482ms，左序为 4.989ms。右序在这棵树上较早形成界。这个对照让我把排序获得的剪枝和产生顺序的代价一起计量。

三分支六层、叶值全为 7 时，α ⩾ β 访问 125 次、读 53 叶；严格 α > β 则为 1093/729，与完整 Minimax 相同。相等已经足以排除后续分支，要求严格超过界会错失节省；两者根值仍正确。

缓存必须区分精确值与界

题三每次调用清表，键为节点对象和玩家，偏移 u=0。Reuse 在精确值、下界达到 β 或上界不高于 α 时成立。其余命中沿用原窗口。Bound 用入口窗，$v \leqslant \alpha_0$ 为上界，$v \geqslant \beta_0$ 为下界，其余为精确。

```
算法 4 TT(s,d,α,β)
if 题四且 Stop(s,d): return Eval(s)
k,u←Key(s,d),Offset(s)
if k in table:
  (z,t)←table[k]; v←z+u
  if Reuse(t,v,α,β): return v
α₀,β₀←α,β
v←AB(s,d,α,β)  # 子递归走 TT
t←Bound(v,α₀,β₀)  # 叶子为 exact
Store(k,v-u,t); return v
```

题三写入保留已有精确值和同类更紧的界。设 X=MAX(6,9)，根为 MAX(MIN(5,X), MIN(X,8))。第一次搜 X 在 β=5 时返回下界 6。误作真值会使根返回 6，正确结果为 8。

固定展开结构与叶值，只改重复引用。强共享访问 958→167，耗时 4.159→1.524ms，纯树仍访问 958 次且 4.332→9.615ms。另一个容量对照中，64 项与不限容量均访问 186 次。重复子树已被覆盖后，扩容未继续减少访问。

Scout 探测省下的求值与重搜代价

¬t 表示换玩家。EVAL 精确求值，TEST 用严格比较短路，q 为真时对应 any，为假时对应 all。并列值无需重搜。

```
算法 5 Scout EVAL(n,t)
if leaf(n): return value(n)
v←EVAL(first(n),¬t)
op←'>' if t=MAX else '<'
for c in rest(n):
  if TEST(c,¬t,v,op): v←EVAL(c,¬t)
return v
TEST(n,t,v,op)
if leaf(n): return value(n) op v
q←(t=MAX)==(op='>')
for c in children(n):
  if TEST(c,¬t,v,op)=q: return q
return ¬q
```

固定顺序并计全部重搜，首候选最佳的 4 组 Scout/同模块 AB 耗时比中位 0.733。镜像后置改进时为 1.980，读叶比 2.900。18 棵随机树左序仅 8 棵更快，重搜成本受首候选质量支配。

题目四 竞争环境下的宝藏分配（综合题）

距离矩阵与博弈状态

A、B 和宝藏格进入费均为 1。以 h=m×Manhattan 做 A*，计算 30 个有序点对，对角为 0。一般数值端点进入费可不同，程序分别计算正反方向。

来源/目标	A	B	T1	T2	T3	T4
A	0	12	10	4	8	8
B	12	0	8	8	4	10
T1	10	8	0	6	6	12
T2	4	8	6	0	6	6
T3	8	4	6	6	0	6
T4	8	10	12	6	6	0

表 7 六个命名位置之间的最小移动费用

完整矩阵中，曼哈顿/零启发扩展 329/650 次，耗时 2.569/3.863ms，费用一致。N 个宝藏需 $O(N^2)$ 次 A*。只存距离占 $O(N^2)$，保留各对格点路径最坏占 $O(N^2V)$。

s=(pA,pB,M,SA,SB,t)，M 是剩余宝藏，t 是行动方。每步只取所选宝藏，已取格仍可通行，对手不挡路。负收益也可选。

```
算法 6 宝藏状态转移
s₀←(A₀,B₀,all,0,0,A)
Actions(s)←[i in M if D[p[t],Ti]<∞]
Next(s,i):
  s'←copy(s); z←Vi-D[p[t],Ti]
  p'[t]←Ti; S'[t]←S[t]+z
  M'←M\{i}; t'←other(t)
  return s'
```

深度减 1 后递归，d=0、M 为空或无可达动作时返回 U=SA−SB。严格改善时用当前动作接上子续走，根部返回首步和 U。N 个宝藏均可达时，第 k 层有 N!/(N−k)!个节点，全深求和为 O(N!)，样例访问 65 次、24 叶。实现每个内部节点扫描 N 位候选，标签序核心时间粗界 $O(N \cdot N!)$。

H2 与 H4 为什么首步不同

两算法同深度根值一致。H2 的 Minimax 访问 17 次，Alpha-Beta 标签/逆/Gain 序分别 13/16/11 次；H4 为 65 次和 47/46/37 次。Gain 序提前收紧界，却要支付排序成本；默认仍按标签访问。

为查清深度为何改变首步，我依次固定 A 的每个选择，再求该深度的最优续走。表中的每个分支另用完整求值核对，包含原搜索中被剪掉的分支。

H	T1	T2	T3	T4	最优首步	最优 U
1	−1	2	−1	−1	T2	2
2	−4	−1	−2	−4	T2	−1
3	−4	2	−1	−4	T2	2
4	−5	1	3	−7	T3	3

表 8 不同深度下四个首步的精确分支值

H2 先取 T2，A 即时收益 6−4=2，B 随后取 T3，收益 7−4=3，U 为−1。H4 的一条最优延续为 T3→T1→T2→T4，收益按各玩家更新后的位置累计：

回合	玩家	移动	价值	路费	当次 Gain	累计 U
1	A	A→T3	7	8	−1	−1
2	B	B→T1	9	8	1	−2
3	A	T3→T2	6	6	0	−2
4	B	T1→T4	7	12	−5	3

表 9 H4 一条最优延续的逐回合收益

沿这条续走，A 得−1，B 得−4，U 为 3。B 最后仍须执行亏 5 的动作。

H2 只计双方第一次行动，未计第二次移动及剩余宝藏竞争；H4 覆盖全部四回合，因此首取 T3 虽即时亏 1，最终相对收益更好。H2 的−1 与 H4 的 3 处在不同截断时点，不能直接把差 4 称为首步策略提高 4。统一用 H4 完整续走评价：浅层首选 T2 的分支值为 1，最优 T3 为 3，固定该首步造成的完整后悔值为 2。

宝藏价值的两个决策边界

样例 v=7 时两种深度选了不同首步。我固定地图及其他价值，只把 T3 价值 v 从 0 扫到 14。H2 首取 T2、T3 的分支值分别为 2−max(1,v−4,−3)和 v−9，在 7.5 相交。H4 四分支为 2−v 、8−v 、v−4 、−v ，最优 U=max(8−v,v−4)，T2/T3 在 v=6 相交。正式 v=7 正好位于两处交点之间。

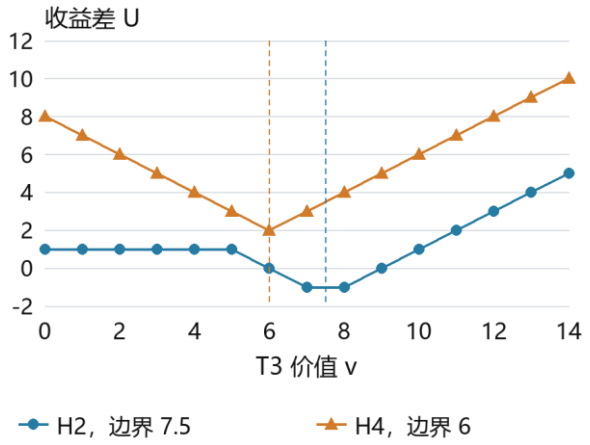


图 3 只改变 T3 价值时的最优收益差。虚线标出 T2 与 T3 的切换点

交点再用两种算法、三种顺序核验。H4 在 v=6 时两首步同为 2，H2 在 v=7.5 时同为−1.5。顺序改变并列首步的选择，最优值相同。

跨地图检验排序与浅层决策

另外生成开放加权、单缺口墙和随机障碍各 10 图，只按连通与命名点互异等结构接收，不按性能筛选。全部距离对照 Dijkstra，H1 至 H4、两算法、三顺序对照完整枚举。

H4 标签序 AB 在 30 图均少于 Minimax 访问，时间却仅 24 图更短。Gain 相对标签序 22 图少访问、3 图相同、5 图更多，时间 19 图快、11 图慢。以同图 H4 完整续走评价浅层首步，标签序 H1/H2/H3/H4 命中最优为 19/19/23/30 图；后悔值中位虽均 0，H2 最大损失达 32。

加深未必逐级改善。例如 open_weighted_s202611104 的 H2/H3/H4 完整后悔值为 1/6/0；H3 截于 A 第二次行动，未计 B 最后取宝，当前 U 更乐观。全部 30 图有 2 图出现 H2 到 H3 变差。

收益增量缓存何时有效

后续动作与已有分数无关，因此 F_d(s)=U+V_d(pA,pB,M,t)。深度 0 时增量 V_0=0。A 对 Gain+子增量取最大，B 对子增量−Gain 取最小，所以相同后续局面的增量相同。六宝藏两条历史的当前 U 为 7/3，终值 10/6，增量均为 3。

算法 4 的键为(pA,pB,M,t,d)，偏移 u=U。只存非终止态的 v−U 和续走。命中时加回当前 U，并取回续走。Minimax 存精确值，Alpha-Beta 按入口窗分类。

双方位置各至多 N+1 种，固定根与 H 时，M 决定轮次和剩余深度，故 K=O(N²2ˆN)。每键只算一次的精确值记忆化需 O(NK)时间和 O(K)值空间。实存长度至 H 的续走，表空间 O(HK)。界不足以截断时仍要重搜，候选排序与距离预计算另计。

四/六/八宝藏的非终止历史与结构状态为 41/41、1237/457、69281/3537。每规模 6 个新输入。八宝藏增量 AB/原 AB 的完整耗时比中位为标签序 0.557、Gain 序 0.750，均 6/6 图更快。四宝藏无命中，六宝藏未稳定回本。H4 标签序 AB 只计博弈搜索时，延迟查询在 31 图均少查；对同统计内核 20 图更快，对原版仅 3 图更快。完整任务还需计入建矩阵。

结论与适用范围

题一的开阔图主要受同 f 排序影响，地标和走廊压缩要靠重复查询摊销准备成本。正式小样例固定 FIFO，曼哈顿较 h=0 少扩展 2 格，时间差不足 0.001ms。

题二先处理免费 E，再结合收紧下界与同分顺序。整数桶缩小后仍慢于原堆。堆清理使峰值 1192→386，固定 ρ 重测 15 块仅 2 块更快，配对耗时差中位数为 +3.769ms，故保留原实现。

题三的好子序可以提早截断。真值排序的预读和 Scout 重搜都占时间，采样 q=8 换 30 棵新树后有 29 棵更慢。共享较多时置换表更合算，容量覆盖常见子树后，继续增大也不再节省访问。

题四用同一 H4 终局比较首步。增量表在八宝藏组两种顺序均改善完整耗时，四宝藏没有重复命中。各规模用了不同地图，因此收益还取决于局面如何重复。我用最优值核对、整次调用的配对时间和新输入复验决定是否采用改动。